Secure Student Record System Design

5655691
WM242-24

TABLE OF CONTENTS

SECTION 1: SYSTEM DESIGN AND JUSTIFICATION

1.1 System Overview……...……[1]

1.1.1 *Organisational Context*……..……[1]

1.1.2 *User Roles and Permissions (RBAC)*...……..……[1]

1.1.3 *Architectural Overview*…….……..……[1]

1.1.4 *Audit Logging and Accountability*……..……[1]

1.2 Choice and Justification of Cryptographic Methods…….……[2]

1.2.1 *Authentication: Argon2 and Multi-Factor Authentication (MFA)*…..…[2]

1.2.2 *Confidentiality: AES-256 (Fernet)*…….……[2]

1.2.3 *Integrity and Non-Repudiation: RSA-PSS Signatures*……[2]

1.2.4 *Key Management and Lifecycle*……..……[2]

1.3 System Usability……..……..……[3]

1.3.1 *Transparent Security*……...……[3]

1.3.2 *Error Handling and Enumeration Prevention*……[3]

1.4 Legal Compliance and Ethical Rules…….……[3]

1.5 Critical Evaluation and Future Improvements……..…….……[3]

1.6 Sequence Diagram……..……...…[4]

1.7 RBAC Matrix…….....……[4]

SECTION 2&3: IMPLEMENTATION AND FUNCTIONAL TESTING

2.1 Identity Management and Argon2id Hashing……[5]

2.1.1 *Implementation Logic*……[5]

2.1.2 *Functional Testing: Brute Force Resistance*……[6]

2.2 Multi-Factor Authentication (MFA)……[6]

2.2.1 *Implementation Logic*……[6]

2.2.2 *Functional Testing: MFA Verification*……[7]

2.3 Data Confidentiality (AES-128 Fernet)……[7]

2.3.1 *Implementation Logic*……[7]

2.3.2 *Functional Testing: Data at Rest*……[8]

2.4 Non-Repudiation (RSA Digital Signatures)……[8]

2.4.1 *Implementation Logic*……[8]

2.4.2 *Functional Testing: Signature Integrity*……[9]

2.5 Key Lifecycle and Rotation……[9]

2.5.1 *Implementation Logic*……[9]

2.6 Robust Error Handling and Exception Management……[10]

2.7 Accountability through Audit Logging……[10]

2.7.1 *Accountability Evidence*……[10]

3.1 Evidence of Legal Accountability……[11]

3.1.1 *Forensic Evidence (CMA 1990)*……[11]

3.1.2 *GDPR Article 32 Compliance*……[11]

3.2 Functional Testing Table……..……[12]

3.3 Critical Evaluation: Scalability and Future Technical Improvements…….……[13]

3.3.1 *From Local JSON to Database Management Systems*……..…[13]

3.3.2 *Transition to Time-based One-Time Passwords (TOTP)*……..……[13]

3.3.3 *Enhanced Logging with ELK Stack*……..……[13]

3.3.4 *Moving to Environment Variables for Key Storage*…….…….[13]

REFERENCES……..……..[14-15]

APPENDIX…….…….[16-31]

SECTION 1: SYSTEM DESIGN AND JUSTIFICATION

1.1 System Overview

1.1.1 *Organisational Context*

As a Junior Security Engineer, I was tasked with designing a secure educational platform to manage sensitive information like student records and academic feedback. The design philosophy is centered on the CIA Triad: ensuring Confidentiality of student grades, maintaining Integrity of the records, and Availability of the system for users while upkeeping strict Non-Repudiation regarding grade changes. [1]

1.1.2 *User Roles and Permissions (RBAC)*

The system enforces the Principle of Least Privilege by implementing Role-Based Access Control (RBAC). [2]

- **Students:** Lowest privilege - only given "Read-Only" access for their own grades.
- **Lecturers:** Medium privilege - allowed to "Read" and "Write" student data.
- **Administrators:** High privilege - Responsible for system maintenance, audit log monitoring and key lifecycle management.

1.1.3 *Architectural Overview*

The system follows a "Defense in Depth" strategy, where all users must initially pass through the centralised authentication gateway before their role is evaluated. The sensitive data is stored at rest in an encrypted format by using Symmetric Encryption, whilst authenticity is guaranteed by Asymmetric Digital Signatures. The attack surface of the login gateway is significantly reduced due to both successful password and MFA code requirements before access to the database is granted. [3]

A Zero Trust model is followed, where the data layer does not automatically trust the user solely because they managed to open the application. A valid cryptographic key is also required, only derived after successful authentication and MFA are passed. This "Gatekeeper" architecture ensures if the UI is bypassed, the data remains encrypted and useless. [4]

1.1.4 *Audit Logging and Accountability*

Post encryption, a secure system must be able to provide Accountability. The implementation includes a `log_event` function that records all activity: successful logins, failed login attempts, grade modifications and key rotations. This satisfies the Accounting part of the AAA framework. In a real-world scenario, these logs would be sent to a write-only server to prevent attackers from being able to remove evidence after a breach. [5]

1.2 **Choice and Justification of Cryptographic Methods**

1.2.1 ***Authentication: Argon2 and Multi-Factor Authentication (MFA)***

I chose Argon2id, the winner of the Password Hashing Competition in July 2015, for password hashing. Unlike the widely adopted SHA-256, a fast general-purpose hash, Argon2 is specifically designed to be memory-hard, providing a second layer of defense against offline "Brute-Force" and GPU-accelerated cracking attacks. [6,7]

This implementation significantly increases the "Work Factor" for an attacker. By making the hash hardware "expensive", brute-force attempts are both economically and technically unfeasible.

To mitigate online attacks, I have also implemented a 5-attempt lockout policy, balancing the protection with user experience. To further strengthen the entry point for the system, I implemented a Multi-Factor Authentication (MFA) layer. Whilst the primary authentication relies on the "Knowledge Factor" of the user's password, the MFA provides a secondary "Possession Factor". This aligns with security principles explored in Lab 8, in which relying solely on a single factor of authentication creates a "single point of failure", and so a multi-layered defense is needed to protect high-value assets like student grades. This ensures even if a password is compromised, the entire system's confidentiality is not at risk.[8]

1.2.2 ***Confidentiality: AES-128 (Fernet)***

As plain AES is difficult to implement manually without making mistakes like using the same Initialisation Vectors twice, I opted to use the Fernet recipe for data confidentiality. [9]

Fernet is built on top of AES-128 in CBC mode with a Hash-based Message Authentication Code (HMAC) using SHA-256. This approach of "Authenticated Encryption" is superior to standard AES, as it ensures even if an attacker cannot read the data, they cannot modify a single bit without the system detecting it through the `InvalidToken` exception, providing a dual layer of Confidentiality and Integrity. [10]

1.2.3 ***Integrity and Non-Repudiation: RSA-PSS Signatures***

To prevent unauthorised modification, the system utilises RSA Digital Signatures. When a Lecturer updates a grade for one of the students, the JSON string is signed with their Private Key, ensuring: [11]

- Integrity: The system can detect if a single character in the grades file has been modified.
- Non-repudiation: A lecturer cannot deny changing a grade as the signature has been produced with their unique private key. [12]

1.2.4 ***Key Management and Lifecycle***

The problem with using one key to encrypt data is more "leakage" occurs, hence a bigger impact on integrity if the key is stolen. To align with NIST SP 800-57 guidelines from Lab 5, a key rotation mechanism is implemented, limiting the Cryptoperiod of the system master key. If a previous key were compromised, the data that was re-encrypted with the current key

remains secure, limiting the "Blast Radius" of a potential credential leak, ensuring the system maintains long-term cryptographic agility. [13]

1.3 System Usability

1.3.1 *Transparent Security*

The security system is "invisible" to the end-user, as decryption happens automatically in the background after a successful login and MFA check. This prevents "Security Fatigue" in which users may attempt to bypass any complex manual procedures.

1.3.2 *Error Handling and Enumeration Prevention*

The system uses generic error messages during login like "Invalid Username or Password", preventing Username Enumeration, in which an attack may use different inputs to identify valid usernames in the database.

1.4 Legal Compliance and Ethical Rules

GDPR Compliance: The system complies with "Privacy by Design" by using encryption by default. Data Minimisation was implemented by only storing hashes of passwords, not the plaintext passwords themselves. [14,15]

Computer Misuse Act 1990: The Audit Logging feature ensures all access and activity is tracked, providing a forensic trail necessary for legal accountability in the event of an internal breach. [16,17]

1.5 Critical Evaluation and Future Improvements

While the current 5-attempt lockout policy can mitigate individual brute-force attacks effectively, a future version of the system should include Global Rate Limiting or Mass Lockout Protection. Currently, an attacker would be able to rotate through thousands of different usernames, incorrectly inputting 4 passwords into each without triggering lockout. By implementing a system-wide threshold, like locking all login attempts if a set number failures occur across the system over a period of time, a defence against distributed credential-stuffing attacks is ensured.

Another future improvement involves the inclusion of Hardware Security Modules (HSMs). Currently, the `secret.key` is stored on the local disk, but in a production environment, the key should be stored on a dedicated hardware device or cloud key management service to ensure even a user with root access cannot export the encryption key.[18]

1.6 **Sequence Diagram**

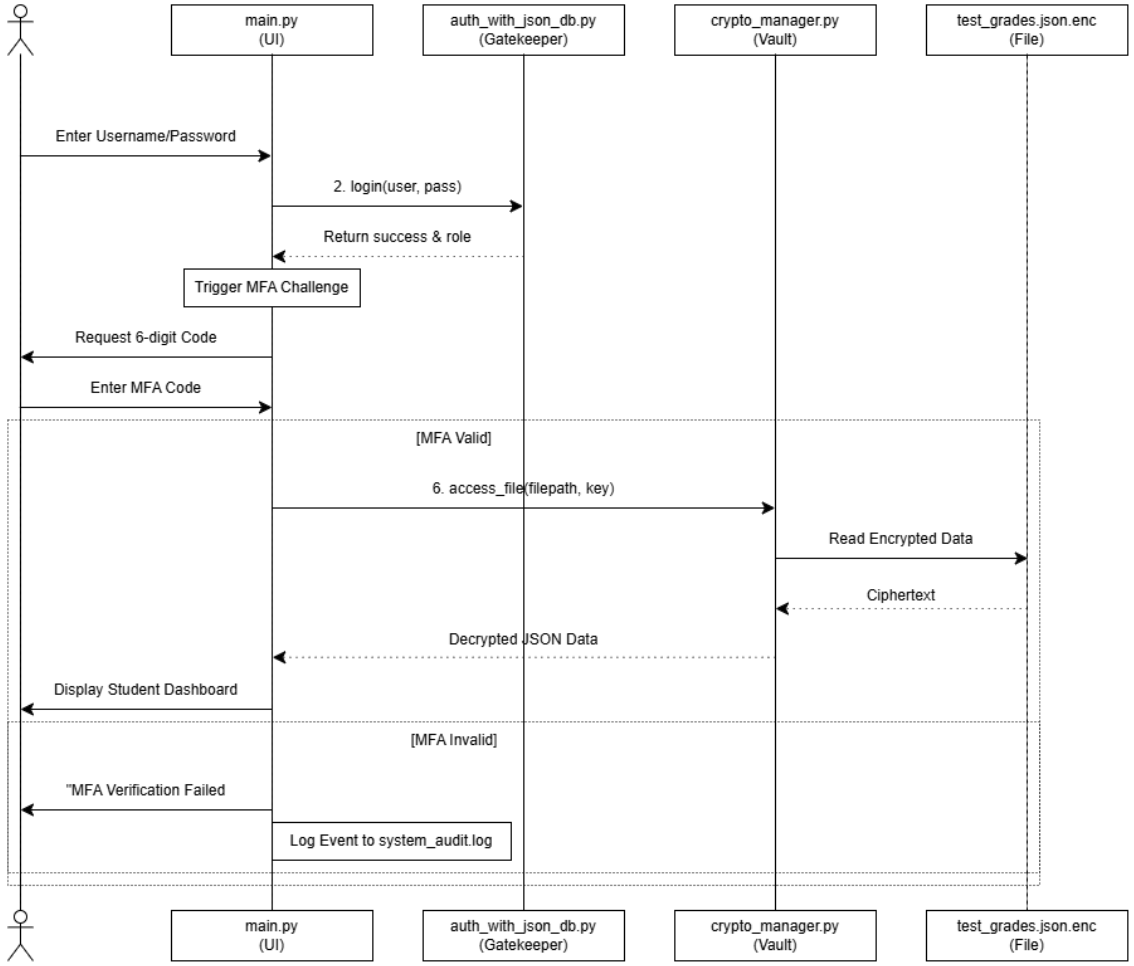


Figure 1: Sequence Diagram

As illustrated in the Sequence Diagram (Figure 1), the authentication process is a multi-step communication process. The `main.py` controller initiates a request to the `auth_with_json.db` module. The system only proceeds to the MFA challenge when a `True` value is returned. This sequential logic ensures no cryptographic keys are loaded into the memory until the user's identity has been fully proved, minimising the system's attack surface and ensures that data stays encrypted at rest until the final gate is cleared.

1.7 **RBAC Matrix**

Role	View Grades	Edit Grades	View Audit Logs	Rotate Keys
Student	Only own	No	No	No
Lecturer	All	Yes	No	No
Admin	All	Yes	Yes	Yes

Figure 2: RBAC Matrix

SECTION 2&3: IMPLEMENTATION AND FUNCTIONAL TESTING

2.1 Identity Management and Argon2id Hashing

2.1.1 *Implementation Logic*

The first line of defense is the authentication module. I implemented the Argon2id algorithm by using the `argon2-cffi` library. Argon2id is the current industry standard as it is "memory-hard", requiring a specific amount of RAM to compute, combatting high-speed GPU cracking attempts which SHA-256 is vulnerable to.This is managed by creating a centralised `login` function, including a key technical feature of the Account Lockout Tracker. By using a dictionary called `failure_tracker`, the system monitors failed attempts in real-time. [7]

```
# From auth_with_json_db.py
def login(user, password):
    # Check lockout status before performing expensive hash verification
    if failure_tracker.get(user, 0) >= 5:
          print(f"Access Denied: '{user}' is locked due to too many wrong
attempts")
        return "Locked"

    try:
        stored_hash = mock_db[user]["hash"]
        ph.verify(stored_hash, password)
        failure_tracker[user] = 0  # Reset on success
        return mock_db[user]["role"]
    except VerifyMismatchError:
        failure_tracker[user] = failure_tracker.get(user, 0) + 1
        return None
```

Technical Analysis:

In this block, the ph.verify() method handles the salts and hashing internally, ensuring even identical passwords produce different hashes in storage, and protects against common attacks like rainbow table attacks. The `try/except` block is an important security control as it catches the `VerifyMismatchError` and increments the failure counter. This prevents "Timing Attacks" by guaranteeing consistent response times for incorrect passwords, and eventually triggering the "Locked" status to prevent brute-forcing. [7]

2.1.2 ***Functional Testing: Brute Force Resistance***

Test Case: Repeatedly entering the wrong password for “mock_student”.

Expectation: The system should block access after the 5th attempt.

Evidence:

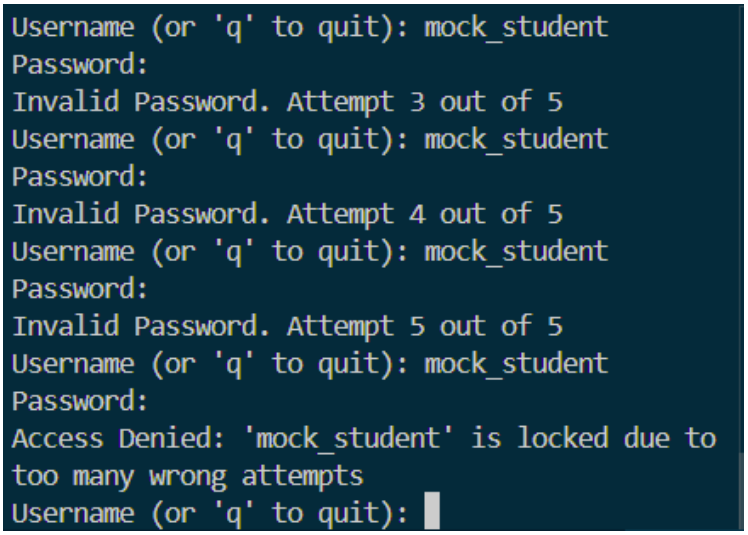


Figure 3: Terminal displaying error message after submitting incorrect password 5 times

2.2 **Multi-Factor Authentication (MFA)**

2.2.1 ***Implementation Logic***

After a successful password check, the system initiates a secondary “Possession Factor” check, handled by `mfa_logic` in `main.py`. A Pseudo-Random Number Generator is used to create a randomised 6-digit token. [8]

```
# From main.py
def mfa_logic(user, role):
    # create mfa code
    mfa_code=str(random.randint(100000,999999))
     print(f"A code has been sent to your device (For testing - mfa code: {mfa_code})")

    user_mfa = input("Enter the code: ")

    if user_mfa != mfa_code:
        print("Mfa verification failed")
        log_event(user, "mfa failure")
        return False

    log_event(user, f"Login and MFA Successful - Role: {role}")
    return True
```

Technical Analysis:

The use of `random.randint` ensures the generated MFA code changes with each login attempt, following the principles of "Ephemeral Credentials", reducing the "window of opportunity for attackers", making stolen credentials useless within minutes. Including this code makes sure the stolen password is not enough to compromise the account. If the verification fails, it is immediately logged to the audit file, implementing the Defense in Depth strategy where the failure of one factor does not lead to a total system compromise. [3]

2.2.2 *Functional Testing: MFA Verification*

Test Case: Correct password but incorrect MFA code.

Expectation: The system denies access and returns to the login prompt

Evidence:

```
Username (or 'q' to quit): mock_student
Password:
A code has been sent to your device (For testing - mfa code: 652840)
Enter the code: 123456
Mfa verification failed
MFA failed, try again
Username (or 'q' to quit):
```

Figure 4: Terminal displaying error message for failed MFA verification

2.3 **Data Confidentiality (AES-256 Fernet)**

2.3.1 *Implementation Logic*

The data layer is protected by symmetric encryption, using Fernet from the `cryptography` library, providing Authenticated Encryption. [9]

```
# From crypto_manager.py
def protect_file(filepath, key):
    f=Fernet(key)
    with open(filepath, "rb") as file:
        data = file.read()
        encrypted_data = f.encrypt(data)
        with open(filepath+".enc","wb") as file:
            file.write(encrypted_data)
```

Technical Analysis:

The `protect_file` function first reads the raw JSON data in binary mode (`rb`). AES-128 in CBC mode is applied and HMAC is appended by the f.encrypt() method. This shows an important security distinction, as standard AES only provides Confidentiality, but Fernet provides Integrity as well. If an attacker modified a single character from the `.enc` file, the decryption process would fail with an `InvalidToken` error, alerting the system to tampering of a protected file. [10]

2.3.2 ***Functional Testing: Data at Rest***

Test Case: Attempting to read the encrypted grades file.

Expectation: Content should be unreadable ciphertext.

Evidence:

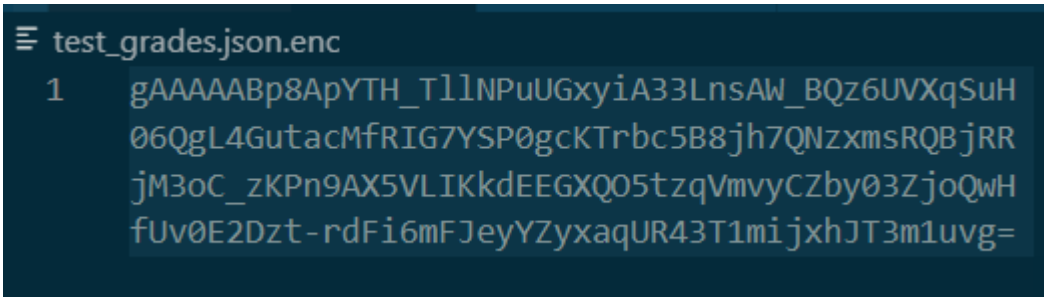


Figure 5: Unreadable ciphertext when opening the “test_grades.json.enc” file

2.4 <u>Non-Repudiation (RSA Digital Signatures)</u>

2.4.1 ***Implementation Logic***

To make sure grades are only updated by the authorised lecturers, RSA Digital Signatures was implemented to provide Non-Repudiation. The lecturer that changed a grade cannot claim otherwise, as the signature is tied to their unique private key. [11]

```
# From crypto_manager.py
def sign_data_system(data_string):
    private_key, _ = load_system_signing_keys()
    signature = private_key.sign(
        data_string.encode(),
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                    salt_length=padding.PSS.MAX_LENGTH),
                    hashes.SHA256()
    )
    with open("grades.sig", "wb") as f:
        f.write(signature)
    return signature
```

Technical Analysis:

The logic uses Probabilistic Signature Scheme (PSS) padding with a SHA-256 hash. By signing the JSON string before it is encrypted, a verifiable link is created between the updated data content and the signer’s identity. Setting the salt length to `MAX_LENGTH` increases the entropy of the signature, making it strong against signature forgery. [12]

2.4.2 ***Functional Testing: Signature Integrity***

Test Case: Manually changing a grade within the file and attempting a signature check.

Expectation: The system returns "Verification failed".

Evidence:

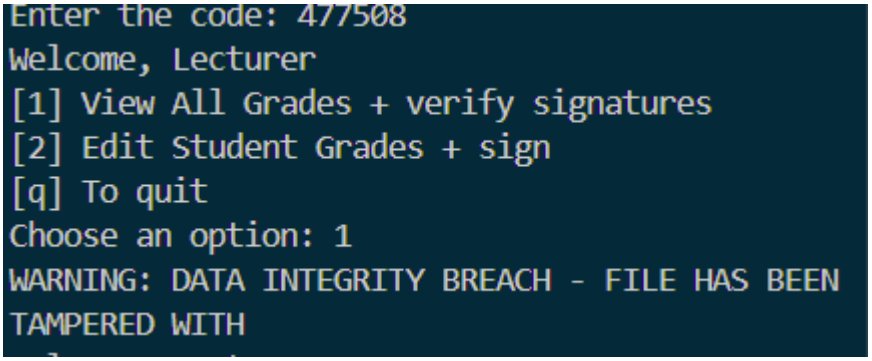


Figure 6: Terminal error message after manually editing a file

2.5 **Key Lifecycle and Rotation**

2.5.1 ***Implementation Logic***

To align with NIST SP 800-57 standards, the "Cryptoperiod" is managed by a rotation function in `crypto_manager.py`. This limits the risk of key exhaustion and also limits the effect on the system if a key is stolen by an attacker. [13]

```
# From crypto_manager.py
def rotate_system_key(filepath):
    # decrypt data with old key
    old_key = load_key()
    raw_data = access_file(filepath, old_key)

    # create and save new key
    new_key = Fernet.generate_key()
    with open(KEY_FILE, "wb") as f:
        f.write(new_key)

    # re-encrypt with new key
    protect_file_raw(raw_data, filepath, new_key)
```

Technical Analysis:

This function performs a critical security maintenance task of "Decipher-then-Encipher" migration. In the `main.py` loop, this option is only available for the Administrator role due to RBAC, ensuring Separation of Duties and preventing the lower-privileged users from compromising the system's core trust mechanisms. [2]

2.6 **Robust Error Handling and Exception Management**

An important part of secure implementation is when the system fails, it closes rather than stays open. In `crypto_manager.py`, comprehensive error handling is implemented by using Python's `try-except` blocks.

```
# From crypto_manager.py logic
try:
    with open("grades.sig", "rb") as f:
        signature = f.read()
    # ... verification logic ...
except FileNotFoundError:
     print("Security Alert: Signature file missing. Data integrity cannot
be verified.")
    return False
except Exception as e:
    print(f"Cryptographic Error: {e}")
    return False
```

Technical Analysis:
By catching the `FileNotFoundError`, the system prevents a scenario in which a missing signature file may be ignored, allowing the possibility of unsigned data being processed. This "Fail-Safe" mechanism ensures if the key, ciphertext or signature is missing or corrupted, the system defaults to a No Access state. This is required for maintaining the Integrity of student records under the GDPR framework. [14]

2.7 **Accountability through Audit Logging**

2.7.1 ***Accountability Evidence***

Every action occurring in the system is recorded in the `system_audit.log`, satisfying the Accounting requirement of the AAA framework. The centralised `log_event` function within `main.py` ensures the system keeps a chronological record of all events, showing the user, action and the time for each session. [5]

```
# From main.py
def log_event(user, action):
    timestamp = datetime.datetime.now().strftime("%d-%m-%Y %H:%M:%S")
    with open("system_audit.log", "a") as f:
        f.write(f"[{timestamp}] User: {user} | Action: {action}\n")
```

Technical Analysis:

The `datetime` library is used to generate timestamps, critical for forensic investigation after a security incident. The file is opened using `"a"` (append) mode instead of `"w"` (write) mode, ensuring the historical ledger remains intact by only adding information, without overwriting existing data.From a security perspective, this function provides Non-Repudiation. By logging every grade modification and login attempt, users are unable to deny their actions. In a production-grade environment, this file would ideally be stored in a remote logging server to prevent a local attacker from clearing the logs to hide their tracks. [5]

3.1 Evidence of Legal Accountability

3.1.1 *Forensic Evidence (CMA 1990)*

The Computer Misuse Act 1990 requires organisations to identify unauthorised access, and the `system_audit.log` provides evidence for such identification. [17]

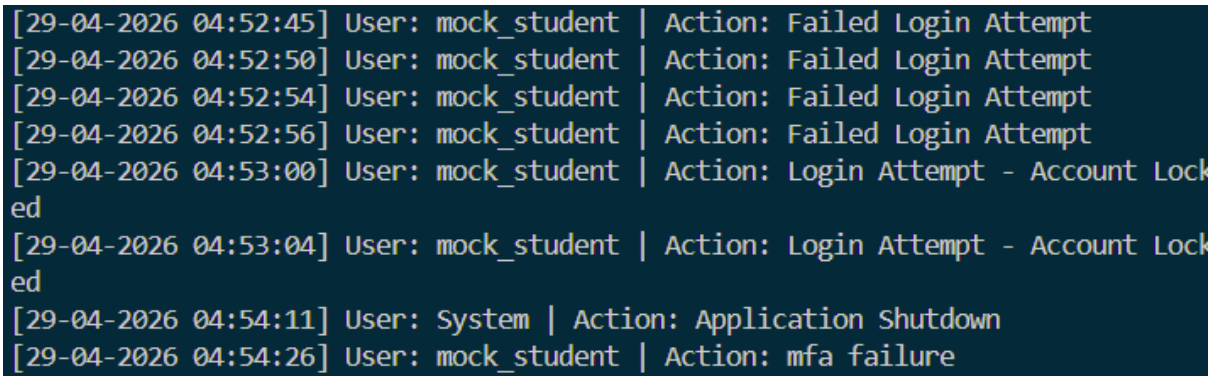


Figure 7: Audit log displaying entries of failed login and MFA attempts

As shown in the screenshot above, the system has successfully captured failed MFA attempts and brute-force lockouts. Since these logs contain the unique username and timestamp, the forensic trail is provided, satisfying the legal requirements of the CMA. It shows that the system actively monitors and blocks unauthorised "intent", a key component of legal accountability.

3.1.2 *GDPR Article 32 Compliance*

GDPR requires "appropriate Technical and Organisational Measures" to protect personal data. [19]

- **Confidentiality Proof:** Test Case TC-04 (below) confirms that the data is unreadable with an incorrect key.
- **Integrity Proof:** Test Case TC-05 confirms that the RSA-PSS signatures successfully detect and block modified data.
- **Accountability:** The audit logs ensure that Data Subject Access Requests (DSARs) are fulfilled by showing who accessed or modified a specific student's grade records.[5]

3.2 **Functional Testing Table**

Test ID	Requirement	Test Type	Action/Input	Expected Result	Actual Result	Status
TC-01	Brute Force	Negative	5x Incorrect Password	Lockout triggered	Account locked	**PASS**
TC-02	Lockout Persistence	Boundary	Try 6th time after lockout	System should refuse attempt immediately	Access denied	**PASS**
TC-03	MFA Integrity	Negative	Correct password + Wrong MFA	Entry to dashboard blocked and event logged	Access Denied	**PASS**
TC-04	Confidentiality	Privacy	Open `.enc` file in VScode	Data must be ciphertext	Unreadable data	**PASS**
TC-05	Integrity (File)	Negative	Modify 1 character in `grades.json.enc`	Fernet raises `InvalidToken` exception	Access Denied	**PASS**
TC-06	RBAC (Student)	Privilege	Login as Student and attempt to edit	Menu option suppressed	Restricted	**PASS**
TC-07	RBAC (Lecturer)	Privilege	Login as Lecturer and attempt to rotate key	Key rotation option hidden/unauthorised	Restricted	**PASS**
TC-08	Key Life	Functional	Admin runs “Rotate Keys”	Master key replaced and file re-encrypted	Key rotated	**PASS**
TC-09	Audit	Forensic	Run any test	Check `system_audit.log`	Log updated	**PASS**
TC-10	Input Validation	Robustness	Enter empty string for Username	System handles without crashing	Invalid Input	**PASS**

Figure 8: Test Cases Table

3.3 **Critical Evaluation: Scalability and Future Technical Improvements**

3.3.1 ***From Local JSON to Database Management Systems***

The current system uses local `local.json` and `test_grades.json` files. While the design works as a prototype, it lacks Concurrency Control. If two lecturers tried to sign and update grades simultaneously, they would cause a "Race Condition" to occur, leading to data corruption or an invalid digital signature. A future implementation would migrate this local database to a PostgreSQL database with row-level locking, allowing the system to scale to thousands of students whilst still maintaining the Availability part of the CIA triad. [1]

3.3.2 ***Transition to Time-based One-Time Passwords (TOTP)***

The current MFA implementation uses pseudo-random integers created when the user logs in. To improve scalability and user experience, it could be upgraded to support RFC 6138 (TOTP). The system could generate seeds that sync with commonly used apps like Google Authenticator by using the `pyotp` library. This removes the need to "send" a code (such as via SMS) which costs money and instead relies on time-synchronisation, reducing operational costs as the user base grows. [20]

3.3.3 ***Enhanced Logging with ELK Stack***

The `log_event` function adds to a local text file. In a large-scale educational environment, these should instead be pushed to an ELK stack (Elasticsearch, Logstash, Kibana), allowing administrators to create real-time dashboards and set up automated alerts for "Threshold Breaches" that indicate a credential-stuffing attack. [5]

3.3.4 ***Moving to Environment Variables for Key Storage***

The current design stores the `secret.key` as a file on the local disk, creating a security risk as an attacker can obtain the master key if they gain read-access to the directory. An improvement for the future would be to store the Fernet key in either an Environment Variable or dedicated Secret Manager like AWS KMS, ensuring the key only exists in volatile memory during runtime. [18]

4: References

[1] NIST (2020) Zero Trust Architecture. NIST Special Publication 800-207. Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://csrc.nist.gov/publications/detail/sp/800-207/final

[2] Sandhu, R.S., Coyner, E.J., Ferraiolo, D.F. and McHugh, J. (1996) 'Role-based access control (RBAC) models and administrative issues', Proceedings of the First ACM Workshop on Role-based Access Control, pp. 1–10.

[3] CIS (2021) CIS Critical Security Controls v8. Center for Internet Security. Available at: https://www.cisecurity.org/controls

[4] NIST (2020) Zero Trust Architecture. NIST Special Publication 800-207. Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://csrc.nist.gov/publications/detail/sp/800-207/final

[5] NIST (2006) Guide to Computer Security Log Management. NIST Special Publication 800-92. Gaithersburg, MD: National Institute of Standards and Technology

[6] OWASP (2023) Password Storage Cheat Sheet. OWASP Foundation. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[7] RFC 9106 (2021) Argon2 Memory-Hard Function for Password Hashing and Cryptographic Key Derivation. IETF. Available at: https://www.rfc-editor.org/rfc/rfc9106.html

[8] NIST (2020) Digital Identity Guidelines: Authentication and Lifecycle Management. NIST Special Publication 800-63B. Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://pages.nist.gov/800-63-3/sp800-63b.html

[9] Roth, K. (2024) Fernet (symmetric encryption). cryptography Documentation. Available at: https://cryptography.io/en/latest/fernet/

[10] NIST (2001) Advanced Encryption Standard (AES). FIPS PUB 197. Gaithersburg, MD: National Institute of Standards and Technology.

[11] NIST (2023) Digital Signature Standard (DSS). FIPS PUB 186-5. Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf

[12] Kaliski, B. (2021) Public-Key Cryptography Standards (PKCS) #1 v2.2: RSA Cryptography Specifications. RFC 8017. IETF. Available at: https://www.rfc-editor.org/rfc/rfc8017.html

[13] NIST (2020) Recommendation for Key Management – Part 1: General. NIST Special Publication 800-57 Part 1 Revision 5. Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://nvlpubs.nist.gov/nistpubs/sp800-57/sp800-57-Part1r5.pdf

[14] European Parliament and Council (2016) Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, L 119, pp. 1–88. Available at: https://gdpr-info.eu/art-25-gdpr/

[15] European Parliament and Council (2016) Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, L 119, pp. 1–88. Available at: https://gdpr-info.eu/art-5-gdpr/

[16] The Stationery Office (1990) Computer Misuse Act 1990. London: HMSO. Available at: https://www.legislation.gov.uk/ukpga/1990/18/contents

[17] Crown Prosecution Service (2020) Computer Misuse Act: Guidance for Prosecutors. London: CPS. Available at: https://www.cps.gov.uk/prosecution-guidance/computer-misuse-act

[18] NIST (2023) Security Requirements for Cryptographic Modules. FIPS PUB 140-3. Gaithersburg, MD: National Institute of Standards and Technology

[19] European Parliament and Council (2016) Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, L 119, pp. 1–88. Available at: https://gdpr-info.eu/art-32-gdpr/

[20] RFC 6238 (2011) TOTP: Time-Based One-Time Password Algorithm. IETF. Available at: https://www.rfc-editor.org/rfc/rfc6238.html

5: Appendix

AI declaration: AI was used with help for spotting syntax errors, proofreading the report and giving suggestions. Each response was critically evaluated and modified before application.

TEST CASES:

TC-01/02:

```
Invalid Password. Attempt 3 out of 5
Username (or 'q' to quit): mock_student
Password:
Invalid Password. Attempt 4 out of 5
Username (or 'q' to quit): mock_student
Password:
Invalid Password. Attempt 5 out of 5
Username (or 'q' to quit): mock_student
Password:
Access Denied: 'mock_student' is locked due to
too many wrong attempts
Username (or 'q' to quit): mock_student
Password:
Access Denied: 'mock_student' is locked due to
```

TC-03:

```
Username (or 'q' to quit): mock_student
Password:
A code has been sent to your device (For testin
g - mfa code: 783560)
Enter the code: 123456
Mfa verification failed
MFA failed, try again
Username (or 'q' to quit):
```

TC-04:

test_grades.json.enc

```
1   gAAAAABp8ApYTH_TllNPuUGxyiA33LnsAW_BQz6UVXqSuH
    06QgL4GutacMfRIG7YSP0gcKTrbc5B8jh7QNzxmsRQBjRR
    jM3oC_zKPn9AX5VLIKkdEEGXQO5tzqVmvyCZby03ZjoQwH
    fUv0E2Dzt-rdFi6mFJeyYZyxaqUR43T1mijxhJT3m1uvg=
```

TC-05:

```
Enter the code: 477508
Welcome, Lecturer
[1] View All Grades + verify signatures
[2] Edit Student Grades + sign
[q] To quit
Choose an option: 1
WARNING: DATA INTEGRITY BREACH - FILE HAS BEEN
TAMPERED WITH
```

TC-06:

```
Welcome, Student
 You can view your assignments
[1] View grades
[q] To quit
```

TC-07:

```
Welcome, Lecturer
[1] View All Grades + verify signatures
[2] Edit Student Grades + sign
[q] To quit
```

TC-08:

Before:

```
secret.key
1    tfIIGnWJ__cy0xJJYy8guSa-3Vje3U-pNd7ZkUEM9xQ=
```

```
Welcome, Admin
[1] View All Grades
[2] Edit Student Grades
[3] View System Audit Log
[4] Rotate Crypto Key
[q] To quit
Choose an option: 4
This will re-encrypt the database
System key rotated and re-encrypted database su
ccessfully
```

After:

```
secret.key
1    I0pIUGBXn5iamLZaZhLfi5Jm88vWcUyNokwVGqqKhag=
```

TC-09:

```
ole: Student
[29-04-2026 04:58:14] User: mock_student | Action: Login Successful - Role: Stu
dent
[29-04-2026 04:59:58] User: mock_admin | Action: Login and MFA Successful - Rol
e: Admin
[29-04-2026 04:59:58] User: mock_admin | Action: Login Successful - Role: Admin
[29-04-2026 05:00:04] User: mock_admin | Action: Performed system key rotation
[29-04-2026 05:00:30] User: mock_admin | Action: Performed system key rotation
```

TC-10:

```
Username (or 'q' to quit):
Password:
Access Denied
```

Code:
Auth_with_json_db.py:

```
#auth with mock db imported from users.AttributeErrorjson

import json
import argon2
from argon2.exceptions import VerifyMismatchError

ph = argon2.PasswordHasher()

failure_tracker={}

#loading db from json
def load_db():
    try:
        with open("users.json","r") as f:
            return json.load(f)

    except FileNotFoundError:
        return{} #returns nothing as no file

def save_db(db):
    with open("users.json", "w") as f:
        json.dump(db, f, indent=4)

#initialise database
mock_db=load_db()




def login(user,password):
    #check if user exists
    if user not in mock_db:
        print("Access Denied")
        return None

    #check lockout status
    if failure_tracker.get(user,0) >=5:
        print(f"Access Denied: '{user}' is locked due to too many wrong
attempts")
        return "Locked"

    try:
        stored_hash = mock_db[user]["hash"]
        ph.verify(stored_hash,password)
```

```
        #reset failure when successful login
        failure_tracker[user]=0

        if ph.check_needs_rehash(stored_hash):
            mock_db[user]["hash"] = ph.hash(password)
            save_db(mock_db)

        return mock_db[user]["role"]

    except VerifyMismatchError:
        failure_tracker[user]=failure_tracker.get(user,0)+1
        print(f"Invalid Password. Attempt {failure_tracker[user]} out of 5")
        return None


if __name__=="__main__":
    #test 1: successful login
    print("Test 1: Successful Student Login")
    role = login("mock_student", "password123")
    print(f"Result: Access Granted as {role}")

    #test 2: incorrect password
    print("Test 2: Incorrect Password")
    login("mock_student", "wrongpassword")

    #test 3: resetting password attempts
    print("Test 3: Resetting Attempts on Successful Login")
    role=login("mock_student", "password123")
    print("Successful Login and Reset of Attempts")

    #test 4: brute force test
    print("Test 4: 5 Attempt Account Lockout")
    for i in range(5):
        login("mock_student", "bruteforceattack") #should show each failed
attempt

    print("Attempting correct password after 5 failures:")
    login("mock_student", "password123") #should still say "Locked"

    #test 5: invalid user
    print("Test 5: Invalid User")
    login("invalid_user", "password123")
```

main.py:

```
#key management
import json
import os
import getpass
import datetime
import random
from cryptography.fernet import InvalidToken
from auth_with_json_db import login
from crypto_manager import access_file, load_key, initialise_key,
protect_file, rotate_system_key, sign_data_system, verify_signature_system,
generate_system_signing_keys


#audit logging
def log_event(user, action):
    timestamp = datetime.datetime.now().strftime("%d-%m-%Y %H:%M:%S")
    with open("system_audit.log", "a") as f:
        f.write(f"[{timestamp}] User: {user} | Action: {action}\n")

def mfa_logic(user, role):
    #create mfa code
    mfa_code=str(random.randint(100000,999999))
    print(f"A code has been sent to your device (For testing - mfa code:
{mfa_code})")

    user_mfa = input("Enter the code: ")


    if user_mfa !=mfa_code:
        print("Mfa verification failed")
        log_event(user, "mfa failure")
        return False

    log_event(user, f"Login and MFA Successful - Role: {role}")
    return True
    ##may need to put directly in code*

def main():

    initialise_key()
    #initialise system wide signing keys if doesnt exist
    generate_system_signing_keys()

    while True: ##Loops round so dont have to run file each time

        user=input("Username (or 'q' to quit): ")
```

```
        if user.lower()=='q':
            log_event("System", "Application Shutdown")
            break
        password = getpass.getpass("Password: ")


        role = login(user, password)


        if role and role != "Locked":
            if mfa_logic(user, role):
                log_event(user, f"Login Successful - Role: {role}")


                if role == "Admin":
                    while True: ####subloop
                        print(f"Welcome, {role}")
                        print("[1] View All Grades")
                        print("[2] Edit Student Grades")
                        print("[3] View System Audit Log")
                        print("[4] Rotate Crypto Key")
                        print("[q] To quit")

                        choice = input("Choose an option: ")
                        if choice == "1":
                            #view test grades using access_file function
                            key = load_key()

                            try:
                                decrypted_data =
access_file("test_grades.json.enc", key)
                                print(f"Student Grades: {decrypted_data}")

                                log_event(user, "Decrypted and viewed
grades")

                            except InvalidToken:
                                print("WARNING: DATA INTEGRITY BREACH - FILE
HAS BEEN TAMPERED WITH")
                                log_event(user, "Data Integrity Failure")

                            except Exception as e:
                                print(f"Error accessing data: {e}")


                        elif choice =="2":
```

```
                            #shows write access lecturer has vs students read
only
                            #feature: logic to reencrypt data

                            #decrypt data to get dictionary


                            key = load_key()

                            try:
                                decrypted_data =
access_file("test_grades.json.enc", key)
                                grades_dict = json.loads(decrypted_data)
#turns string into python dictionary

                                #update grades
                                name = input("Student name: ")
                                new_grade = input("New grade: ")
                                #validation
                                if not name or not new_grade:
                                    print("Name and grade cannot be blank")
                                    continue

                                grades_dict[name] = new_grade

                                updated_json_string=json.dumps(grades_dict)
                                sign_data_system(updated_json_string)

                                print("Digital signature generated")

                            except FileNotFoundError:
                                print("No file found. Creating new one")
                                grades_dict = {}
                            except Exception as e:
                                print(f"Error accessing data: {e}")




                            #save to temp plane file then protect again
                            with open("temp_update.json", "w") as f:
                                json.dump(grades_dict, f)

                            protect_file("temp_update.json", key)

                            #replace old file with new file
                            os.replace("temp_update.json.enc",
"test_grades.json.enc")
```

```
                            os.remove("temp_update.json")
                            print("Grades updated and database successfully
re-encrypted")


                            log_event(user, f"Modified grade for student:
{name}")

                        #open audit
                        elif choice == "3":
                            with open("system_audit.log", "r") as f:
                                print(f.read())

                        #rotating key
                        elif choice == "4":
                            print("This will re-encrypt the database")
                            try:
                                rotate_system_key("test_grades.json.enc")
                                print("System key rotated and re-encrypted
database successfully")
                                log_event(user, "Performed system key
rotation")
                            except Exception as e:
                                print(f"Rotation failed: {e}")



                        elif choice =='q':
                            log_event("System", "Application Shutdown")
                            break



                elif role == "Student":
                    print(f"Welcome, {role} \n You can view your
assignments")
                    while True:
                        print("[1] View grades")
                        print("[q] To quit")

                        choice=input("Choose an option: ")
                        if choice == "1":
                            key = load_key()

decrypted_data=access_file("test_grades.json.enc", key)
                            grades_dict=json.loads(decrypted_data)

                            if user in grades_dict:
```

```
                            print (f"Your grade: {grades_dict[user]}")
                        else:
                            print("No grades found")

                    elif choice == 'q':
                        break


            elif role =="Lecturer":

                #generate rsa keys for session - now using keys from
crypto manager
                pass

                while True: #sub loop
                    print(f"Welcome, {role}")
                    print("[1] View All Grades + verify signatures")
                    print("[2] Edit Student Grades + sign")
                    print("[q] To quit")

                    choice = input("Choose an option: ")
                    if choice == "1":
                        #view test grades using access_file function
                        key = load_key()

                        try:
                            decrypted_data =
access_file("test_grades.json.enc", key)

                            #verify signature
                            if verify_signature_system(decrypted_data):
                                print("Data is authentic and signed by
lecturer")
                            else:
                                print("Signature does not match data")


                            print(f"Student Grades: {decrypted_data}")

                            log_event(user, "Decrypted and viewed
grades")

                        except InvalidToken:
                            print("WARNING: DATA INTEGRITY BREACH - FILE
HAS BEEN TAMPERED WITH")
                            log_event(user, "Data Integrity Failure")
                        except Exception as e:
                            print(f"Error accessing data: {e}")
```

```
                    elif choice =="2":
                        #shows write access lecturer has vs students read
only
                        #feature: logic to reencrypt data

                        #decrypt data to get dictionary


                        key = load_key()

                        try:
                            decrypted_data =
access_file("test_grades.json.enc", key)
                            grades_dict = json.loads(decrypted_data)
#turns string into python dictionary

                            #update grades
                            name = input("Student name: ")
                            new_grade = input("New grade: ")

                            if not name or not new_grade:
                                print("Name and grade cannot be blank")
                                continue

                            grades_dict[name] = new_grade

                            #turn dict into string
                            updated_json_string = json.dumps(grades_dict)

                            #sign before encryption
                            sign_data_system(updated_json_string)
                            print("Digital signature generated for
update")

                            #save to temp plane file then protect again
                            with open("temp_update.json", "w") as f:
                                f.write(updated_json_string)

                            protect_file("temp_update.json", key)

                            #replace old file with new file
                            os.replace("temp_update.json.enc",
"test_grades.json.enc")
                            os.remove("temp_update.json")
```

```
                                print("Grades updated and database
successfully re-encrypted")


                                log_event(user, f"Modified grade for student:
{name}")

                            except Exception as e:
                                print(f"Error accessing data: {e}")







                        elif choice=='q':
                            log_event("System", "Application Shutdown")
                            break


            else:
                print("MFA failed, try again")
                continue

        elif role == "Locked":
            log_event(user, "Login Attempt - Account Locked")
            continue
        else:

            log_event(user if user else "Unknown", "Failed Login Attempt")
            continue

                ##have some logic to loop round so doesnt logout each time, eg
can view and update grades

main()
```

crypto_manager.py:

```
#aes-256
#hmac-sha256
import os
from cryptography.fernet import Fernet, InvalidToken
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding, rsa


KEY_FILE = "secret.key"

#generates key only if doesnt already exist
def initialise_key():
    if not os.path.exists(KEY_FILE):
        key=Fernet.generate_key()
        with open(KEY_FILE, "wb") as f:
            f.write(key)

#retrieves key
def load_key():
    return open(KEY_FILE, "rb").read()

#encrypts file and replaces original
def protect_file(filepath, key):
    f=Fernet(key)
    with open(filepath, "rb") as file:
        data = file.read()

        encrypted_data = f.encrypt(data)
        with open(filepath+".enc","wb") as file:
            file.write(encrypted_data)

    #remove unencryoted original
    #os.remove(filepath)


def run_crypto_tests():
    initialise_key()

    key=load_key()

    print(f"System intialised. Key loaded from {KEY_FILE}")
```

```
    ###TESTING
    #DUMMY TEST FILE
    with open("test_grades.json", "w") as f:
        f.write('{"Alvin":"F", "Simon":"A", "Theodore":"D"}')

    #protect the file
    protect_file("test_grades.json", key)
    print("File encrypted successfully as test_grades.json.enc")



if __name__=="__main__":
    run_crypto_tests()


#decrypt file for viewing
def access_file(encrypted_path, key):
    f=Fernet(key)
    with open(encrypted_path, "rb") as file:
        encrypted_data = file.read()

    #return original data or show error if tampered with
    return f.decrypt(encrypted_data).decode()


#digital signatures

SYSTEM_PRIVATE_KEY_FILE = "system_private.pem"
SYSTEM_PUBLIC_KEY_FILE = "system_public.pem"

def generate_system_signing_keys():

    #generate if keys dont exist
    if not os.path.exists(SYSTEM_PRIVATE_KEY_FILE):
        print("Generating system-wide signing keys")

        #generates rsa keys for digital signatures
        private_key = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
        public_key = private_key.public_key()

        #save private key
        with open(SYSTEM_PRIVATE_KEY_FILE, "wb") as f:
            f.write(private_key.private_bytes(
                encoding=serialization.Encoding.PEM,
                format=serialization.PrivateFormat.PKCS8,
                encryption_algorithm=serialization.NoEncryption()
            ))
```

```
        #save public key
        with open(SYSTEM_PUBLIC_KEY_FILE, "wb") as f:
            f.write(public_key.public_bytes(
                encoding=serialization.Encoding.PEM,
                format=serialization.PublicFormat.SubjectPublicKeyInfo
            ))

        print(f"Keys saved to {SYSTEM_PRIVATE_KEY_FILE} and
{SYSTEM_PUBLIC_KEY_FILE}")

        return private_key, public_key

    else:
        print("System keys already exist")
        return load_system_signing_keys()

def load_system_signing_keys():
    #load priv key
    with open(SYSTEM_PRIVATE_KEY_FILE, "rb") as f:
        private_key=serialization.load_pem_private_key(
            f.read(),
            password=None
        )
    #load pub key
    with open(SYSTEM_PUBLIC_KEY_FILE,"rb") as f:
        public_key = serialization.load_pem_public_key(
            f.read()
        )
    return private_key, public_key


def sign_data_system(data_string):
    #sign grades string to prove authenticiyy
    private_key, _ = load_system_signing_keys()
    signature = private_key.sign(
        data_string.encode(),
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                    salt_length=padding.PSS.MAX_LENGTH),
                    hashes.SHA256()
    )

    #save sig to file
    with open("grades.sig", "wb") as f:
        f.write(signature)
    return signature

#check grades havent been changed
```

```
def verify_signature_system(data_string):
    try:
        _, public_key = load_system_signing_keys()
        with open("grades.sig", "rb") as f:

            signature=f.read()
        public_key.verify(
            signature,
            data_string.encode(),
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        return True
    except Exception as e:
        print(f"Verification failed: {e}")
        return False




#key rotation
def rotate_system_key(filepath):
    #decrypt data
    old_key = load_key()
    raw_data=access_file(filepath, old_key)

    #create and save new key
    new_key=Fernet.generate_key()
    with open(KEY_FILE, "wb") as f:
        f.write(new_key)

    #re encrypt new key
    temp_file = "temp_rotate.json"
    with open(temp_file, "w") as f:
        f.write(raw_data)

    protect_file(temp_file, new_key)
    os.replace(temp_file + ".enc", filepath)
    os.remove(temp_file)
    return new_key
```